

جامعة أم القرى
UMM AL-QURA UNIVERSITY

SE2203
Software Requirements Engineering

Lecture 8:
Functional Requirements

Recap time

- Framework is a generic means for
- structuring information [template, standard, …]

- Examples:
  - Structured text templates for single requirements
  - Vision & Scope templates
  - Cockburn template for Use Cases
  - VOLERE template for SRS
  - IEEE standard for SRS
  - ISO/IEC/IEEE 29148 standard for SRS
  - …

Learning outcomes for Lecture 8

- What are functional requirements?
- What is difference between Use Cases and Functional Requirements?
- What writing style should we use for Functional Requirements?
- What should we avoid while writing Functional Requirements?



Requirements Specification

- Bridge between (non-technical) stakeholder and software development techniques
- “Know your audience”

Less details required

Software Requirements

More details required

- Customers are extensively involved
- Developers have considerable domain experience



- Customers are less involved
- Project team members are geographically dispersed
- Development will be outsourced
- Testing will be based on requirements

How much detail do you need?

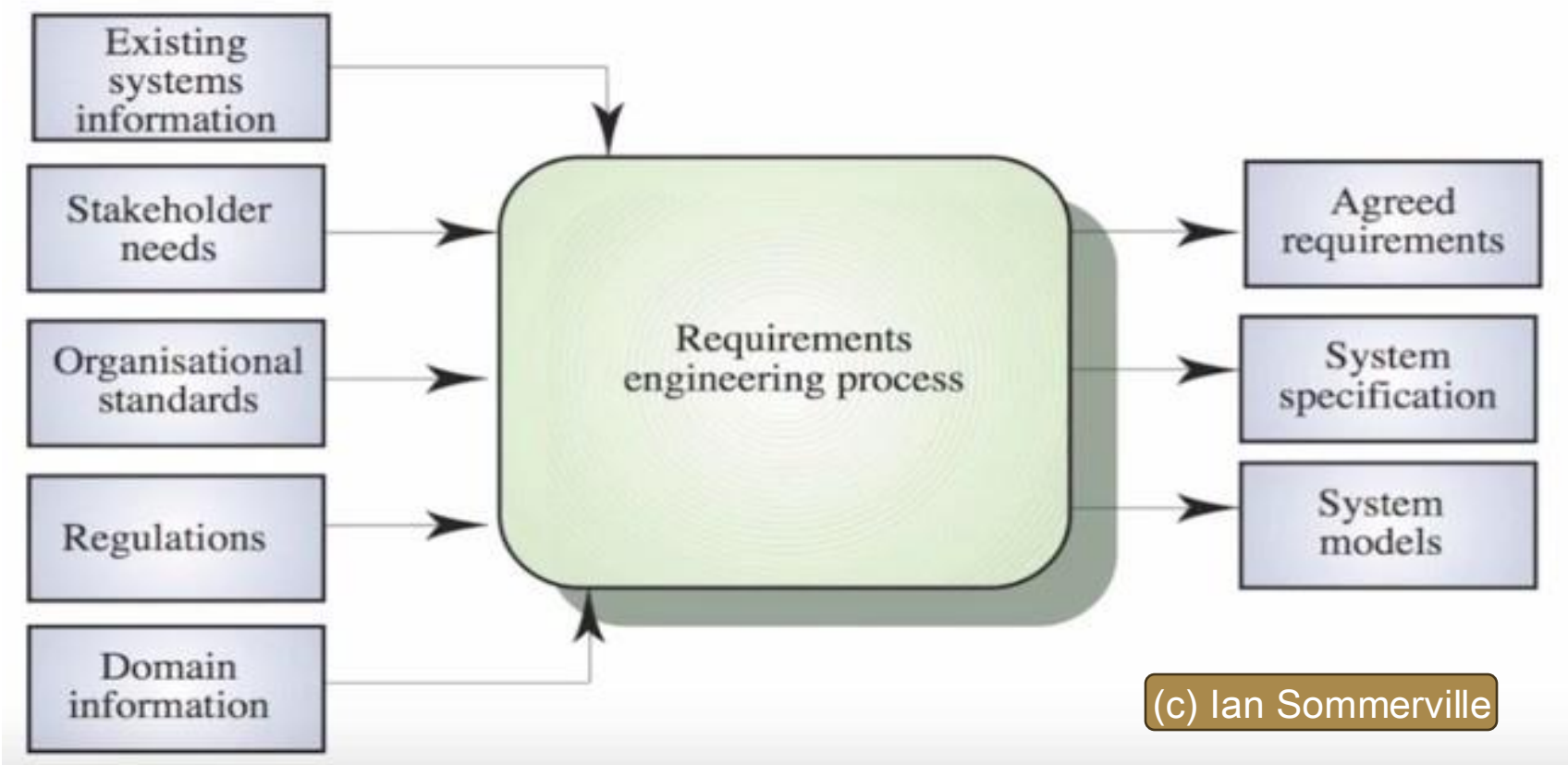
- You need to balance cost and risk
- It costs more to develop requirements in greater details than to leave them more general, but about the overall project costs?
- Choosing the appropriate amount depends upon the risk associated with leaving decisions about specific to developers
- Are you happy to leave the choice to the developer?
- **Murphy's law:**
  - “Whatever can go wrong, will go wrong”
- **Variants on the rule have been formulated:**
  - “If there is a wrong way to do something, then someone will do it”
  - “If there's more than one way to do a job, and one of those ways will result in disaster, then somebody will do it that way.”

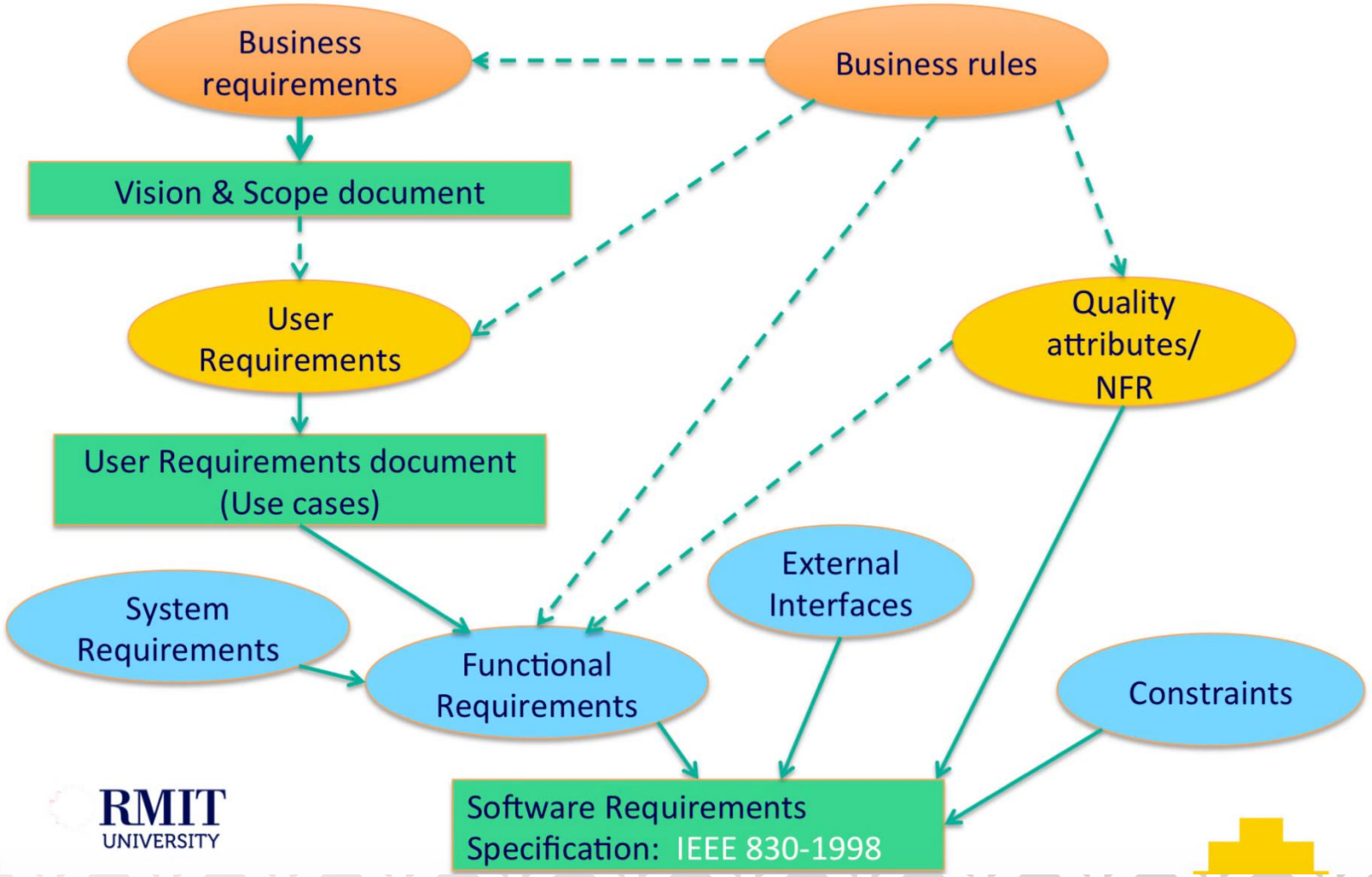
What is influenced by RE?

- Roles involved:
  - Requirements engineer
  - Tester
  - Architect, Designer, Front & Back--end Developer
  - Business Analyst
  - Project Manager
  - Product Owner
  - Domain Expert

- Design, implementation, and integration
- Specification of test cases
- Verification and acceptance within the QA
- Coordination among stakeholders
- System and user documentation
- Contract negotiation
- Project planning
- Evolution
- Budget

What is influenced by RE?




Business requirements
Business rules
Vision & Scope document
User Requirements
Quality attributes/ NFR
User Requirements document (Use cases)
External Interfaces
System Requirements
Functional Requirements
Constraints
Software Requirements Specification: IEEE 830-1998

Types of requirements: System and Functional Requirements

- **System Requirements** describe
  - the top-level requirements for a product that contains subsystems
- **Functional Requirements describe**
  - What functionality the system should have
  - == What the developer is supposed to build
- **Sometimes called behavior requirements**
- **Are presented**
  - within a Software Requirements Specification (SRS)
  - as “shall” statements that describe what the user “shall do” or what the system “shall let the user do”
- External interfaces between system and the outside world include interfaces to other software components, hardware devices, human users, communication interfaces and protocols used to exchange information with other systems

Use Cases vs. Requirements

UCs do not suit all types of software systems, e.g., with time-triggered functionality, computationally intensive applications, etc.

- **Use Cases**
  - Are not requirements!
  - Identify users and goals
  - Way to explore and discover requirements
  - Organized to suit users, not developers: Users may not feel comfortable with functional requirements
  - Work well for interactive end-user applications
  - Often more information is needed than a use case provides

For example:
Use cases are useful for the customer interaction in a flight reservation system, but not in calculating the fare. Calculation of fares are driven by complex business rules, not by the user interaction

From Use Cases to Functional Req.

- Use cases represent ‘higher level of abstraction’
- Analyze use case to derive functional requirements
- Analyze normal and alternative flows for missing functionality
- Determine what happens when a pre-condition is not satisfied
- Detect missing post-conditions
- Trace functional requirements back to a particular use case or cases

But how to write the Functional Requirements?

Requirements: Writing style

- Use a single active sentence
- One sentence per requirement
- Keep sentences short and simple
- Use a present tense verb
- “Shall” is a traditional word used in a functional requirement
  - Must, should, could, would, is recommended, is desirable, is requested, will, …
  - Be consistent whatever your choice
- ‘Disciplined’ English …

- **Good requirements CHECKLIST:**
  - Correct and complete
  - Unambiguous
  - Consistent
  - Specific
  - Measurable and testable
  - Achievable
  - Traceable
  - Prioritized
  - Necessary
- Functional requirements can be written from the system or user perspective

Why bad functional requirements happen to good people?

جامعة أم القرى
UMM AL-QURA UNIVERSITY

- Lack formal training on writing requirements
- Using unconstrained natural language:
  - Introduces ambiguity, vagueness and subjectivity
  - Not always clear, concise and coherent
  - Often not testable
  - Sometimes missing triggers
  - Logic is not always complete (“if” but no “else”)
- “Copy and paste” poor requirements, which multiplies the number of requirements with defects

System Perspective

- **Conditions**: ‘`When [some conditions] …
  - **Result**: “.. the system shall [do something] …”
  - **Qualifier**: “…[response time goal or quality objective]”



- **Example:**
  - When the user indicates that he does not wish to order any more food items, the system shall display all food items ordered, the individual food item prices, and the total payment amount within 1 second.

User Perspective

- **User Type**: “The [user class or actor name]…”
- **Result type**: “… shall be able to [do something] …” Object: “… [to something] …”
- **Qualifier**: “… [response time goal or quality objective]”

- **Example**:
  - The user shall be able to order any meal they had ordered within the previous six months, provided that all food items in that order are available on the menu for the meal date

Be consistent!

Also possible:
- “The system shall let (or allow, permit, enable) the user to [do something]”
- “The system shall provide the ability for the user to [do something]”

Hierarchy: Parent and Child Requirements

- The parent requirement is satisfied by implementing its children
- Find in this example a conflict between parent & child requirement!
- **Example**:
  - **R3.4:** The requestor shall enter a charge number for each chemical ordered
  - **R3.4.1** The system shall validate charge numbers against the master corporate charge number list. If the charge number is invalid, the system shall notify the requestor and shall not accept the order
  - **R3.4.2:** The charge number entered shall apply to an entire order, not to individual line items in the order

How to solve the issue?

- These potential conflicts disappear if the parent requirement is written in the form of a heading, instead of in the form of a functional requirement

- **Example:**
  - **R3.4:** Charge numbers
  - **R3.4.1:** The requestor shall enter a charge number for each chemical in an order.
  - **R3.4.2:** The system shall validate charge numbers against the master corporate charge number list. If the charge number is not found on this list, the system shall notify the requestor and shall not accept the order.

Ambiguity



Sources of ambiguity:

1. Complex logic
   - Don't have lots of if-then-else and nesting
   - Use multiple requirements instead
   - Keep things 'flat'
2. Negative requirements
   - Remove double and triple negatives in all circumstances
   - Changing a negative requirement into a positive one often requires inserting the word only to identify the conditions that permit the system action being described to take place
3. Synonyms
4. Adverbs
5. Wordiness
   - Do not take more words than necessary to make your point
   - Avoid redundant expressions or phrases

What is wrong with the below requirements?

Functional requirement	Ambiguity defect
The software shall support a water level sensor.	What does “support” mean?
The thesaurus software shall display about five alternatives to the requested word.	Boundary values – how many is “about five” 3, 4,6,10 more? Under what conditions are they displayed?
Accepted transactions are posted to the database later.	When is later?
Delinquent accounts are reviewed periodically.	When is periodically?



Removing Negation from Functional Req.

Before	After
The registration process will default to International English and will not present a localized experience until country and language are selected.	The registration process shall default to International English. If the user selects the country and language, the registration process shall present a localized experience.
A domain name cannot be transferred to another registrar during the registration grace period.	The domain administrator may transfer a domain name to another registrar only after the registration grace period.
Training rewards and points will not be visible to users who cannot participate in training rewards.	The system shall display training rewards and points only to users who are permitted to participate in training rewards.

The word **“only”** is a typical way to do this (but not the only one)

Omissions

- **Omissions include:**
  - Missing stimulus to trigger a requirement
  - Missing boundaries
  - How to handle exceptions
  - Missing symmetry or a reverse operation (e.g. collapsing a folder tree should be possible to expand also; undo/redo)
- **Example:**
  - “The system shall generate an error report and forward it to the user”.
  - What is missing?

The requirements does not identify the stimulus

Ambiguity: Synonyms and Adverbs

- Synonyms: Confusion can be caused by using
  - Different terms to refer to the same item.
  - Different terms with subtly different meanings.
  - **Solution:**
    - Consistently use one term
    - Place definitions in a project glossary to align the use of terms
- Adverbs: Words that end in **-ly** often are ambiguous
- Example:
  - Provide a reasonably predictable end--user experience
  - Offer significantly better download times
- Adverbs to use with caution: reasonably, significantly, quickly, broadly, approximately, appropriately, possibly, clearly, effectively, generally, timely, expediently, frequently, directly, easily, ideally, instantaneously, normally, optionally, periodically, preferably, rapidly, transparently, typically, usually

EARS

- Easy Approach to Requirements Syntax (EARS)
  - Created by Alistair Mavin and others from RollsRoyce
  - Presented at the 2009 Requirements Engineering (RE 09) conference

- [optional trigger] [optional precondition] Actor Action [Object].

- Example:
  - When an Order is shipped and Order Terms are not “Prepaid”, the system shall create an Invoice.

EARS

EARS (Easy Approach to Requirements Syntax), Alistair Mavin et al, 17th IEEE InternaDonal Requirements Engineering Conference (RE 09),

- Case study involved Rolls-Royce group working on aircraft engine control systems
- Software was safety critical, contained thousands of components and involved up to twenty different suppliers
- Rolls-Royce identified 8 major problems with existing natural language requirements:
  1. Ambiguity
  2. Duplication
  3. Wordiness
  4. Vagueness
  5. Complexity
  6. Omission
  7. Inappropriate implementation
  8. Untestability

Rewriting requirements using EARS "…demonstrated a significant reduction in all eight problem types…"

- Rewriting requirements using EARS "…demonstrated a significant reduction in all eight problem types…"

EARS: Pros and Cons

- **Strengths:**
  - Simple templates – easy to learn and apply
  - General guidance – provides consistency
  - No tool is needed [but you can use the RE tools, e.g., DOORs following EARS]
  - One of the common forms of requirements communication

- **Weaknesses:**
  - Might be unsuitable for very complex requirements, where non-textual notation might be required, e.g., truth tables

EARS Requirements

- Differentiates between five types (or patterns) of requirements:
  - Ubiquitous (always occurring)
  - Event-driven (“When”)
  - State-driven (“While”)
  - Optional features (“Where”)
  - Complex
  - Unwanted behaviors

- Ubiquitous functional requirements are seldom!
- Most requirements are not ubiquitous
- Things that may seem universal are often subject to unstated triggers or preconditions

Ubiquitous Requirements:
- State a fundamental system property
- Do not require a stimulus in order to execute

EARS Patterns

Pattern type	Pattern name
Ubiquitous	The <system name> shall <system response>
Event-Driven	WHEN <trigger> <optional precondition> the <system name> shall <system response>
Unwanted Behavior	IF <unwanted condition or event>, THEN the <system name> shall <system response>
State-Driven	WHILE <system state>, the <system name> shall <system response>
Optional Feature	WHERE <feature is included>, the <system name> shall <system response>
Complex	(combinations of the above patterns)

Ubiquitous or Not?

- The app should warn the user that the battery level is low
- The installer software shall be available in German.
- The game app should mute all other sound sources in the smartphone.

Ubiquitous or Not?

- The control system shall prevent engine overspeed
- The car shall be compliant with the safety requirements defined in XXX.
- The smartphone should have a mass of no more than XXX grams.
- The software shall prevent Unauthorized Access to patient data.

EARS: Event-Driven Requirements

- Are initiated when and only when a trigger occurs or is detected (at the system boundary)
- The trigger must be something that the system itself can detect

WHEN <trigger> <optional precondition> the <system name> shall <system response>

Examples:

- When the smartphone is off and the power button is pressed, the smartphone shall boot up.
- When the smartphone's battery becomes below 20% charge, the smartphone shall switch to the power--saving mode.

EARS: State-Driven Requirements

- Requirement is active while the system is in a defined state

WHILE <system state>, the <system name> shall <system response>

Examples:

- While the ignition is on, the car system shall display the fuel level to the driver.
- While the aircra1 is in-flight, the control system shall maintain engine fuel flow above XX lbs/sec.

EARS: Optional Features

- Applicable only in systems that include particular feature
- These requirements often will be ubiquitous, but only for systems that include this feature

WHERE <feature is included>, the <system name> shall <system response>

Examples:

- Where the laptop is an “air” model, the laptop shall have a mass of no more than XXX grams.

EARS: Unwanted Behavior

- A variation of event-driven requirement – unwanted events
- Forces separation of
  - Precondition (situation, in which the requirement can be invoked),
  - Trigger (initiating event), and
  - Response (system behavior)

IF <unwanted condition or event>, **THEN** the <system name> shall <system response>

Examples:

- If incorrect PIN code is entered, the smartphone shall display the warning message XXX.
- If the measured and calculated speeds vary by more than 10%, then the software shall use the measured speed.

EARS: Complex requirements

- Describe complex conditional events involving multiple triggers, states and/or optional features
- Use a combination of the keywords When, If/Then, While and Where

<Multiple Conditions>, the <system name> shall <system response>

- Examples:
  - While the aircraft is on-ground, when reverse thrust is commanded, the control system shall enable deployment of the thrust reverser.
  - While the aircra1 is in-flight, if reverse thrust is commanded, then the control system shall inhibit thrust reverser deployment.

Questions?

